

PROJECT REPORT

FIELD SERVICE MANAGEMENT PLATFORM

A web-based platform for managing customers, service sites, technicians, work orders, managers and operational reports.

Internship Organization	Zidio Development
Project	Field Service Management Platform
Role	Java Full Stack Developer Intern
Prepared by	Deepak
Deployment	Vercel + Render + Cloud Database
Report Date	19 August 2026

Confidentiality note: The report contains representative application screenshots. Personal email addresses and other sensitive-looking sample data should be treated as demonstration data.

Report Contents

1	Abstract
2	Introduction
3	Problem Statement
4	Objectives
5	Scope of the Project
6	Technologies Used
7	System Architecture
8	Project Modules
9	Database Design
10	Implementation
11	Testing
12	Screenshots / User Interface
13	Deployment
14	Results
15	Advantages
16	Future Enhancements
17	Conclusion
18	References

1. Abstract

The **Field Service Management Platform** is a full-stack web application designed to organize and streamline field-service operations through a single digital interface. The platform provides secure user authentication and centralized management of customers, service sites, technicians, managers and work orders. It also provides an operational dashboard, notifications, upcoming-task visibility and reporting features so that service information can be viewed and managed efficiently.

The application follows a modern client-server architecture. A responsive React and TypeScript frontend communicates with a Java Spring Boot backend through REST APIs. Authentication is protected using token-based security, while operational data is maintained in a MySQL-compatible relational database. The application has been deployed using Vercel for the frontend and Render for the backend, demonstrating an end-to-end cloud deployment workflow.

The completed system demonstrates practical full-stack development skills including UI development, REST API integration, authentication, database-driven CRUD operations, role-oriented navigation, reporting and production deployment.

2. Introduction

Field-service organizations handle a continuous flow of customer requests, service locations, technicians and maintenance work orders. When these activities are managed through disconnected records or manual processes, information can become difficult to track and operational decisions can be delayed.

This project addresses that operational challenge by providing a centralized web platform. Authorized users can sign in, view operational information from a dashboard, maintain customer and site records, manage technicians and managers, create and update work orders, and review service performance through reports.

The project was developed as part of the Zidio Development internship and focuses on delivering a usable, database-driven application rather than an isolated prototype.

3. Problem Statement

Field-service activities require coordination between customers, service sites, technicians, managers and work orders. Without a centralized system, teams may face difficulty maintaining consistent records, identifying current work-order status, assigning technicians and obtaining a quick operational overview.

The problem is therefore to develop a secure and easy-to-use platform that consolidates these activities and makes important service information available through a unified interface.

4. Objectives

- Develop a centralized web application for field-service operations.
- Provide secure sign-in and account-creation functionality.
- Manage customer information and service locations through CRUD operations.
- Maintain technician and manager information in a structured manner.
- Create, update, assign and track work orders with priorities and statuses.
- Provide a dashboard containing operational KPIs, recent work orders and activity information.

- Provide reporting and analytics features for operational review.
- Connect the frontend, backend and relational database through REST-based communication.
- Deploy the application so that the completed system can be accessed through a live URL.

5. Scope of the Project

The project scope covers the core workflow required to manage field-service operations from a browser-based application. The implemented scope includes authentication, customer management, site management, technician management, manager directory, work-order management, dashboard monitoring, reports and system settings.

Area	Covered Functionality
Authentication	Sign in, create account, password visibility, remember-me option and access-controlled application entry.
Customers	Search, filter, view, add, edit and delete customer records.
Sites	Manage service locations and associate sites with customers.
Technicians	Maintain technician contact information, skills, availability/status and assigned-order count.
Work Orders	Create and manage service requests with title, description, priority, status, site and technician assignment.
Managers	Maintain a directory of management contacts and their departments/status.
Reports	Review operational metrics and service-performance visualization and export reports.
Settings	Configure company information and platform display preferences.

6. Technologies Used

Layer	Technology	Purpose
Frontend	React, TypeScript, Vite, CSS	User interface, navigation, forms and API integration
Backend	Java, Spring Boot	REST APIs, business logic and server-side processing
Security	JWT-based authentication / Spring Security	Secure API access and authenticated sessions
Database	MySQL-compatible relational database	Persistent storage of application data
Version Control	Git, GitHub	Source-code versioning and repository management
Deployment	Vercel, Render	Production hosting for frontend and backend

7. System Architecture

The application uses a layered client-server architecture. The browser hosts the React/TypeScript user interface. The frontend communicates with the Spring Boot backend using REST APIs. The backend applies authentication and business rules before reading or modifying data in the relational database.

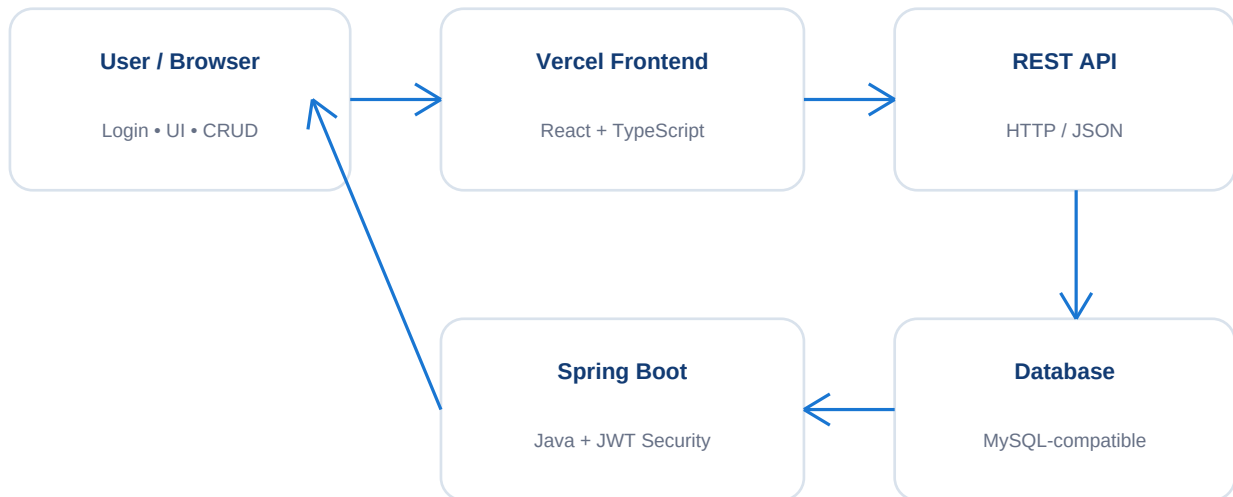


Figure 1. High-level system architecture.

Request flow: User → React/TypeScript UI → REST API → Spring Boot services/security → relational database → response returned to the UI.

8. Project Modules

Authentication Module — Provides sign-in and account creation. The login interface includes password visibility, remember-me and password-recovery entry points. Authenticated users are taken into the operational dashboard.

Dashboard Module — Provides a consolidated operational view containing customer/site/work-order/technician KPIs, recent work orders, quick actions, recent activity, notifications, upcoming tasks and schedule status.

Customer Management — Provides a searchable customer table with customer ID, name, email, phone, city and state, together with view, edit and delete actions.

Site Management — Maintains service locations and their relationship with customers. The interface provides search/filter controls and site creation/editing actions.

Technician Management — Maintains technician records including contact details, skills, status and assigned-order count. This supports workforce visibility and work-order assignment.

Work Order Management — Manages service work orders with title, description, priority, status, site and assigned technician. The list view makes active and completed work easy to identify.

Manager Directory — Provides a centralized directory of management contacts with department, email, phone and status information.

Reports & Analytics — Provides operational metrics such as completed jobs, pending review and average resolution, along with a service-performance visualization and report export actions.

System Settings — Provides company information fields and platform preferences such as theme and compact-layout options.

9. Database Design

The application uses a relational database so that operational information can be stored consistently and queried by the backend. The major business concepts visible in the application are users/authentication, customers, sites, technicians, work orders and managers.

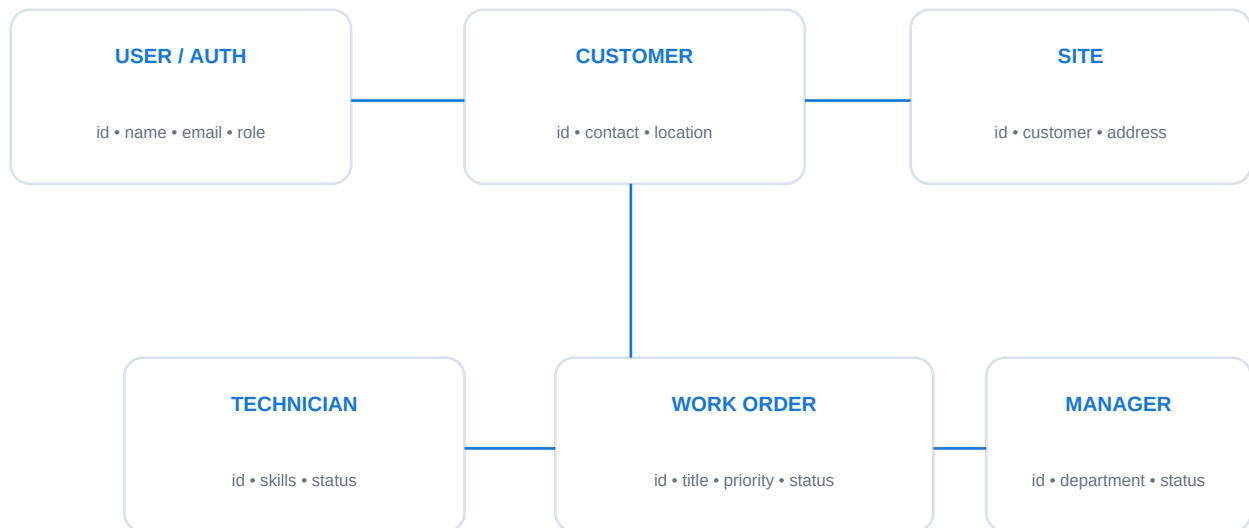


Figure 2. Logical data model used to explain the main application entities.

The key relationship pattern is that customers can have service sites, work orders are associated with service sites and may be assigned to technicians, while user authentication provides controlled access to the application. The report intentionally presents a logical model rather than reproducing implementation-specific database columns.

10. Implementation

The frontend is organized around application pages such as Dashboard, Sites, Work Orders, Customers, Technicians, Managers, Reports and Settings. Reusable forms and tables support data entry and management operations, while the API service layer handles communication with the backend.

The backend is implemented with Java Spring Boot. REST controllers expose application operations, service classes handle business logic, and security components process authenticated requests. JWT-based authentication is used to protect application access and API requests.

The project is maintained in a Git repository with separate frontend and backend areas, database resources, deployment configuration and project documentation. The final repository contains the main application source code and deployment configuration required for the submitted system.

11. Testing

Functional testing was performed through the live application workflow. The following representative scenarios cover the major user-facing capabilities demonstrated by the project.

Test Area	Scenario	Expected Result
Authentication	Sign in with valid credentials	User enters the authenticated application/dashboard.

Registration	Open Create Account and submit account details	Account-creation workflow is available for new users.
Customers	View/search customer records	Customer records are displayed with management actions.
Sites	Open site management and add-site workflow	Sites can be viewed and associated with customers.
Technicians	View technician list	Technician status and assigned-order information are visible.
Work Orders	Create/view/update work orders	Work orders show priority, status, site and technician assignment.
Dashboard	Review operational summary	KPIs, recent orders, activity and task information are displayed.
Reports	Open Reports & Analytics	Metrics and service-performance visualization are displayed.
Settings	Open system settings	Company information and display preferences are available.
Deployment	Open production URL	The deployed frontend loads and communicates with the backend.

12. Screenshots / User Interface

The following screenshots were captured from the deployed application and demonstrate the main user journeys and functional modules.

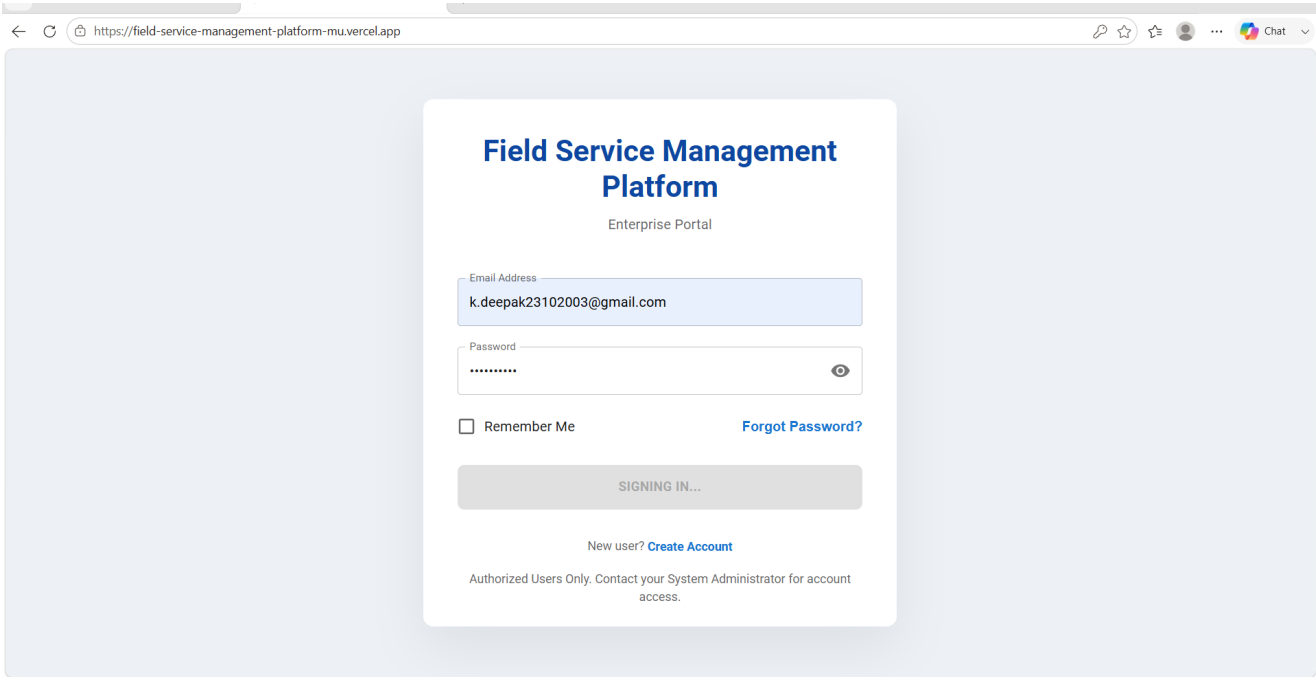


Figure 3. Sign-in page with secure authentication entry points.

←↻🔒https://field-service-management-platform-mu.vercel.app/register🔍🌟👤⋮🗨️Chat

Field Service Management Platform

Enterprise Portal — Create an account

Full Name *

Deepak

Email Address *

k.deepak23102003@gmail.com

Password *

••••••••

👁️

Confirm Password *

••••••••

Role

CUSTOMER

▼

⌂

Already have an account? [Sign In](#)

Figure 4. Create Account page for new users.

←↻🔒https://field-service-management-platform-mu.vercel.app/dashboard🔍🌟👤⋮🗨️Chat

Field Service Management Platform

Deepak

CUSTOMER

🏠 Dashboard

📍 Sites

📦 Work Orders

👤 Customers

🔧 Technicians

👥 Managers

📊 Reports

⚙️ Settings

WELCOME BACK,

Deepak

Wednesday, August 19, 2026

Total Customers

4

Customer accounts tracked

Total Sites

2

Active service locations

Active Work Orders

1

Currently in progress

Completed Orders

1

Completed work orders

Pending Orders

0

Open work orders

Technicians

3

Registered technicians

Recent Work Orders

Latest work order summary from live field service database.

VIEW ALL

WO ID	Title	Site	Priority	Status	Technician	Date
-------	-------	------	----------	--------	------------	------

Quick Actions

Create Customer

Create Site

Create Work Order

Figure 5. Dashboard overview showing operational KPIs and quick actions.

The screenshot displays the 'Field Service Management Platform' dashboard. On the left is a sidebar with navigation links: Dashboard, Sites, Work Orders, Customers, Technicians, Managers, Reports, and Settings. The main content area is divided into three sections. The top section, 'Recent Work Orders', features a table with columns: WO ID, Title, Site, Priority, Status, Technician, and Date. It lists three work orders: WO-5 (washing machine, HIGH priority, IN_PROGRESS status, Unassigned technician), WO-4 (AC Maintenance Test Updated, HIGH priority, ASSIGNED status, Deepak technician), and WO-2 (AC Repair, MEDIUM priority, COMPLETED status, Deepak technician). A 'VIEW ALL' button is in the top right of this section. The bottom right section, 'Recent Activity', shows a timeline of events: 08:55 PM for WO-5, 08:56 PM for WO-4, and 11:52 AM for WO-2, each with a brief description of the activity.

WO ID	Title	Site	Priority	Status	Technician	Date
WO-5	washing machine	Chennai Head Office	HIGH	IN_PROGRESS	Unassigned	8/12/2026
WO-4	AC Maintenance Test Updated	Chennai Head Office	HIGH	ASSIGNED	Deepak	8/8/2026
WO-2	AC Repair	Chennai Head Office	MEDIUM	COMPLETED	Deepak	7/31/2026

Figure 6. Dashboard recent work-order table and recent activity.

This screenshot shows the same dashboard with a different layout. The 'Recent Work Orders' table is no longer visible. Instead, the main area contains three primary sections: 'Notifications' on the left, 'Upcoming Tasks' in the center, and 'Schedule Overview' on the right. The 'Notifications' section includes a 'High Priority Alert' (2 high-priority work order(s) requiring attention), a 'Pending Dispatch Queue' (0 open work order(s) ready for service assignment), and a 'Technician Resource Status' (3 active field technician(s) registered in database). The 'Upcoming Tasks' section lists three tasks: WO-5 (washing machine), WO-4 (AC Maintenance Test Updated), and WO-2 (AC Repair), all at the Chennai Head Office. The 'Schedule Overview' section provides an overview of operational assignments and site dispatch status, showing 0 Open Queue, 1 In Progress, and 2 Service Sites, with a total of 3 Technicians.

Figure 7. Dashboard notifications, upcoming tasks and schedule overview.

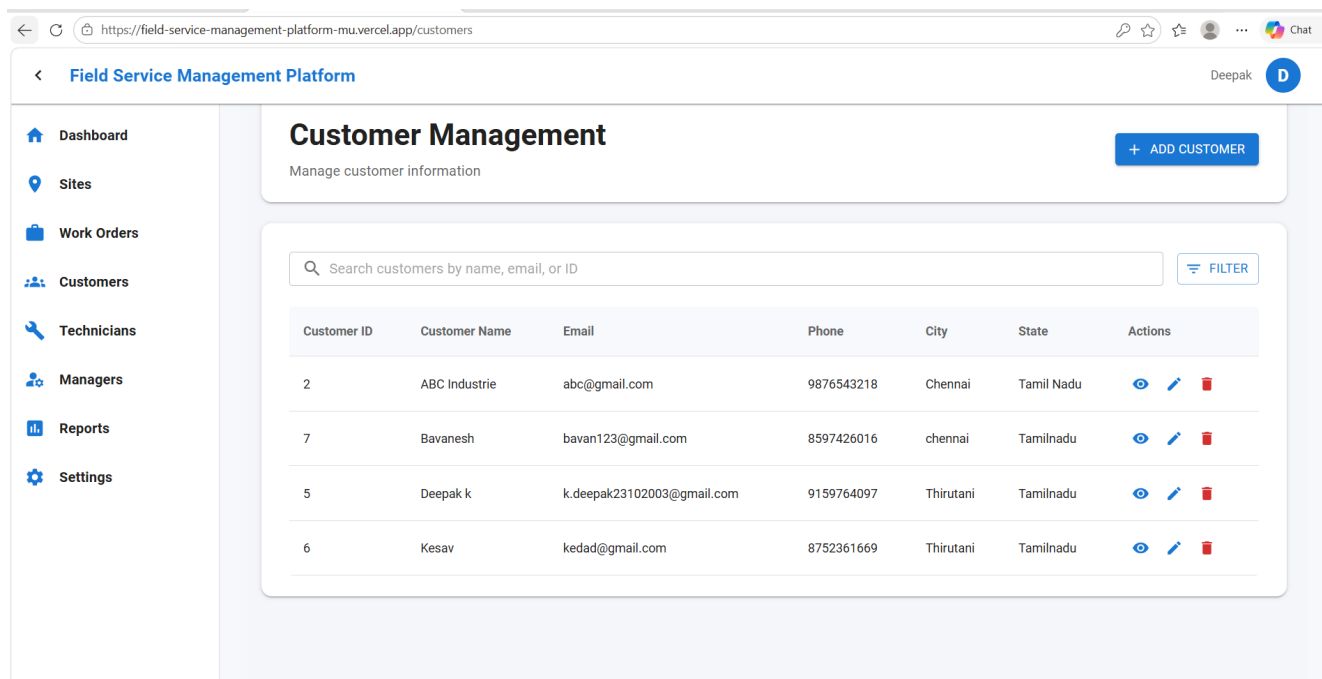


Figure 8. Customer Management module with search, filter and CRUD actions.

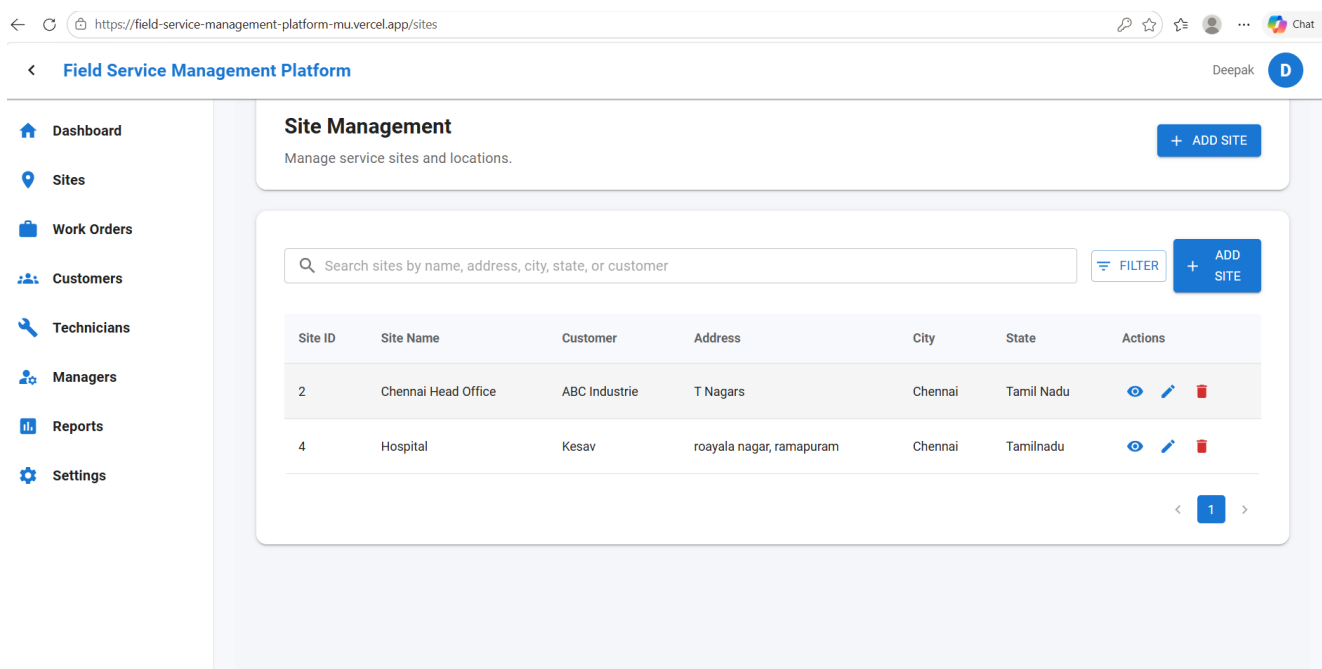


Figure 9. Site Management module with service locations and customer associations.

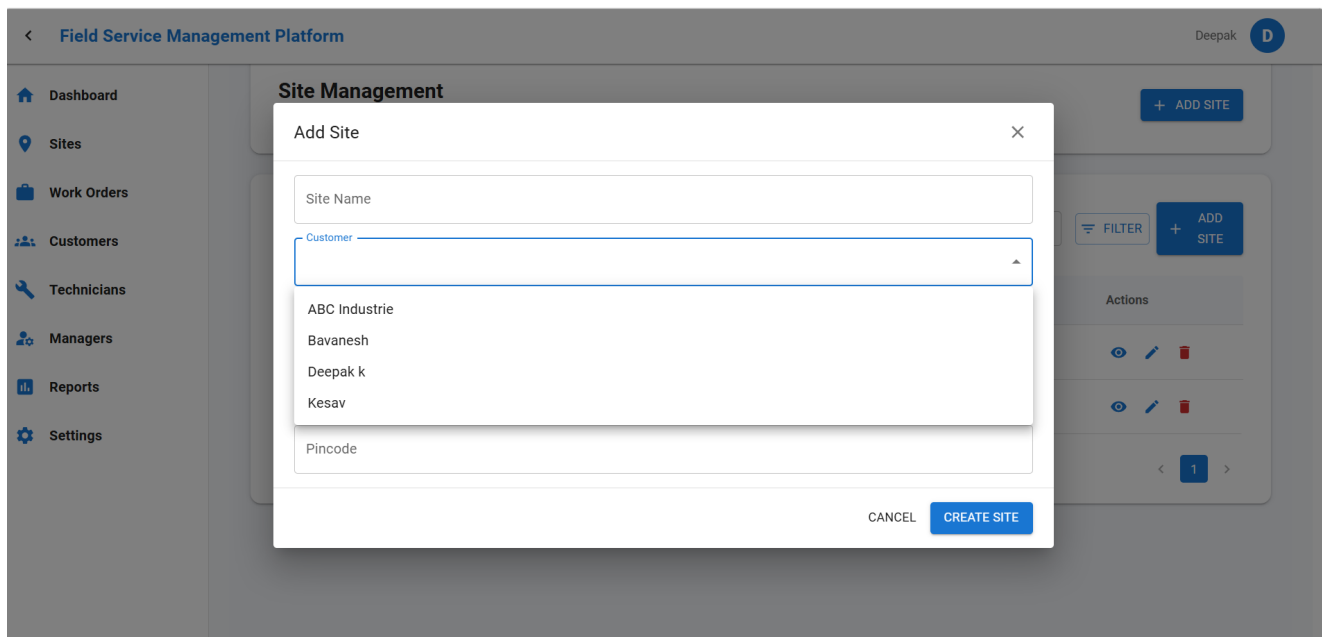


Figure 10. Add Site workflow with customer selection.

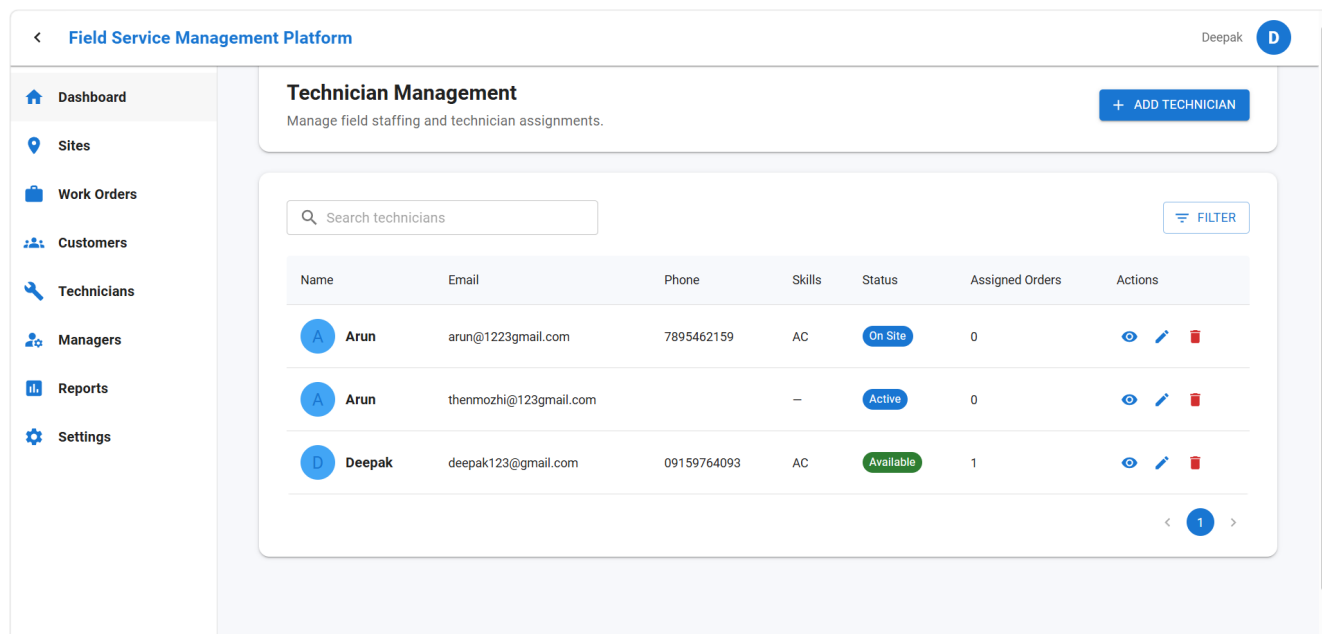


Figure 11. Technician Management module showing skills, status and assigned orders.

The screenshot shows the 'Work Order Management' module. The left sidebar contains navigation links: Dashboard, Sites, Work Orders, Customers, Technicians, Managers, Reports, and Settings. The main content area has a header 'Work Order Management' with a subtitle 'Manage work orders, assignments, priorities, and statuses.' and an '+ ADD WORK ORDER' button. Below this is a search bar 'Search by title, site, technician, or status' and a 'FILTER' button. A table displays the following data:

Work Order ID	Title	Site	Priority	Status	Assigned Technician	Date	Actions
5	washing machine	Chennai Head Office	HIGH	IN_PROGRESS	Unassigned	8/12/2026	
4	AC Maintenance Test Updated	Chennai Head Office	HIGH	ASSIGNED	Deepak	8/8/2026	
2	AC Repair	Chennai Head Office	MEDIUM	COMPLETED	Deepak	7/31/2026	

At the bottom right of the table is a pagination control showing '< 1 >'.

Figure 12. Work Order Management module showing priority, status and technician assignment.

The screenshot shows the 'Add Work Order' form. The form has the following fields:

- Title (text input)
- Description (text input)
- Priority (dropdown menu, currently set to 'Medium')
- Status (dropdown menu, currently set to 'Open')
- Site (dropdown menu)
- Assigned Technician (dropdown menu)

At the bottom right of the form are two buttons: 'CANCEL' and 'CREATE WORK ORDER'.

Figure 13. Add Work Order form.

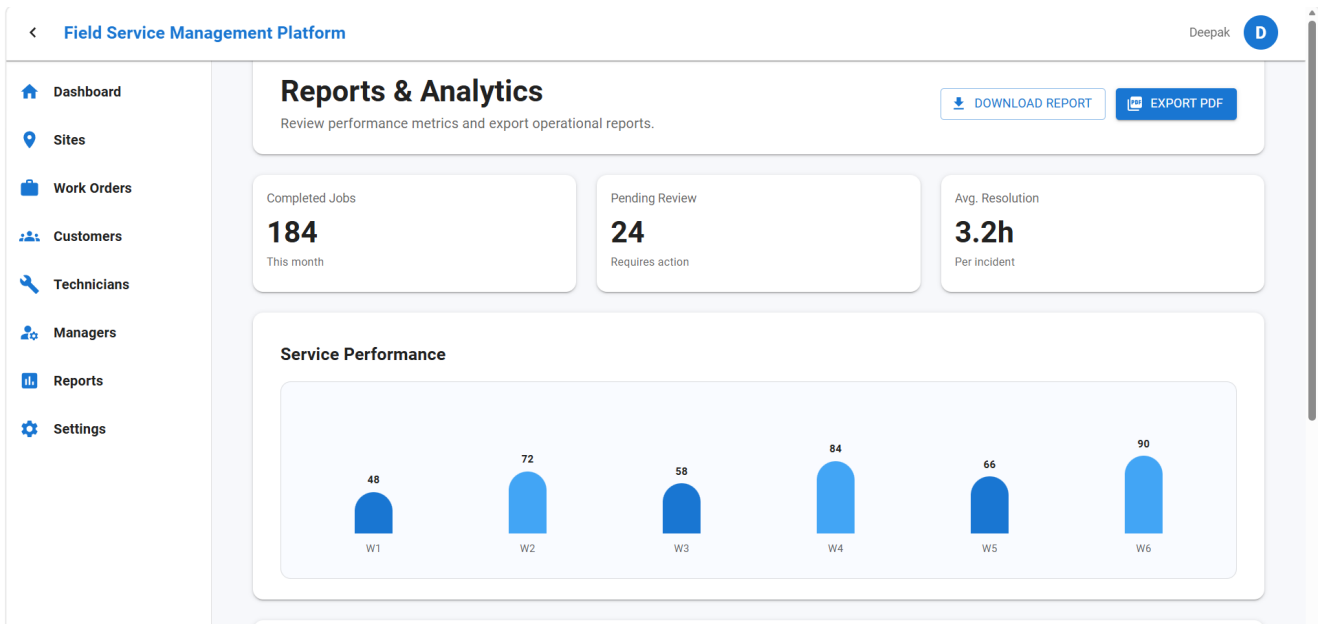


Figure 14. Reports & Analytics module with operational metrics and service performance.

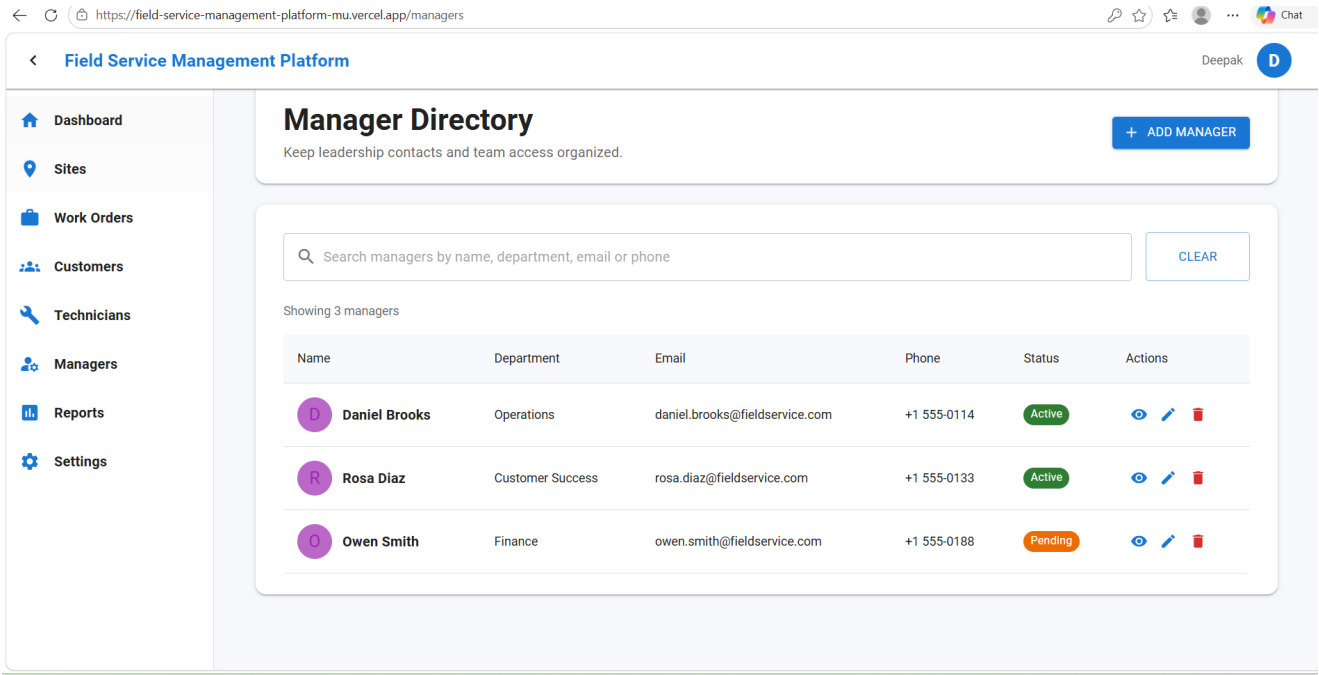


Figure 15. Manager Directory with department, contact and status information.

Figure 16. System Settings for company information and display preferences.

13. Deployment

The project was prepared as a production-deployed full-stack application. The frontend is hosted on Vercel, while the Spring Boot backend is hosted on Render. The application uses a cloud-hosted relational database and communicates through configured production API endpoints.

Component	Deployment / Resource	Status
Source Code	GitHub repository	Public repository / main branch
Frontend	Vercel	Production deployment
Backend	Render	Production deployment
Database	Cloud relational database	Connected to backend
Live Application	https://field-service-management-platform-mu.vercel.app	Accessible production URL

Repository: <https://github.com/deepak-12381/field-service-management-platform>

14. Results

The completed implementation provides a unified operational interface for field-service management. The deployed application demonstrates the ability to authenticate users, display live operational information and manage core service entities.

- A centralized dashboard provides a quick operational summary.
- Customer and service-site information can be managed through dedicated modules.
- Technician availability/status and assigned-order information can be reviewed.
- Work orders can be represented with priorities, statuses, sites and technician assignments.
- Reports & Analytics provides an operational performance view.
- The frontend and backend have been deployed as a connected production system.

- The source code and deployment configuration are maintained in a GitHub repository.

15. Advantages

- Centralized management of field-service information.
- Simple browser-based interface with dedicated operational modules.
- Faster visibility into current work-order status and technician assignment.
- Reduced dependency on disconnected manual records.
- Search and filtering support for operational lists.
- Dashboard and analytics improve operational visibility.
- Cloud deployment makes the application accessible outside the development environment.

16. Future Enhancements

- Role-specific dashboards and permissions for administrators, managers, technicians and customers.
- Email/SMS/push notifications for assignment, status changes and scheduled service.
- Technician mobile application with location-aware job updates.
- Calendar-based scheduling and route optimization.
- Advanced analytics with trend reports, SLA tracking and technician performance metrics.
- File/image attachments for work orders and service evidence.
- Automated backups, audit logs and stronger production monitoring.

17. Conclusion

The Field Service Management Platform successfully demonstrates the development of a complete full-stack business application for service operations. It brings authentication, customer management, site management, technician management, work-order tracking, manager information, reporting and system settings into a single platform.

The project also demonstrates an end-to-end engineering workflow: designing the user interface, implementing REST-based backend services, securing application access, connecting persistent data, managing source code with Git, and deploying the application to cloud platforms. The result is a practical and extensible foundation that can be enhanced with scheduling, notifications, mobile support and advanced analytics.

18. References

- Zidio Development — Internship project context and submission requirements.
- React documentation — <https://react.dev/>
- TypeScript documentation — <https://www.typescriptlang.org/docs/>
- Spring Boot documentation — <https://spring.io/projects/spring-boot>
- Spring Security documentation — <https://spring.io/projects/spring-security>
- Vercel documentation — <https://vercel.com/docs>
- Render documentation — <https://render.com/docs>
- Git documentation — <https://git-scm.com/doc>
- GitHub repository — <https://github.com/deepak-12381/field-service-management-platform>

- Live application — <https://field-service-management-platform-mu.vercel.app>

End of Project Report

Field Service Management Platform • Zidio Development Internship